1. Illustrating Life Cycle of a Thread

```
public class Main {

    static class MyThread extends Thread {

        public void run() {
            System.out.println("Thread is in RUNNING state.");

            try {
                System.out.println("Thread is going to SLEEP.");
                Thread.sleep(1000);

                System.out.println("Thread is RUNNING again.");
            } catch (InterruptedException e) {
                System.out.println("Thread Interrupted.");
            }
        }
    }

    public static void main(String[] args) throws Exception {

        MyThread t = new MyThread();
        System.out.println("Thread State after creation: " + t.getState());
        t.start();
        System.out.println("Thread State after start(): " + t.getState());
        Thread.sleep(200);

        System.out.println("Thread State during sleep: " + t.getState());
        t.join();
        System.out.println("Thread State after completion: " + t.getState());
    }
}
```

OUTPUT:

```
Thread State after creation: NEW
Thread State after start(): RUNNABLE
Thread is in RUNNING state.
Thread is going to SLEEP.
Thread State during sleep: TIMED_WAITING
Thread is RUNNING again.
Thread State after completion: TERMINATED
```

2. Thread Priorities

```
class MyThread extends Thread {
```

```
    MyThread(String name) {
        super(name);
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(
                getName() + " - Priority: " + getPriority()
                + " - Count: " + i
            );
        }
    }
}
public class Main {
    public static void main(String[] args) {
        MyThread high = new MyThread("High Priority");
        MyThread medium = new MyThread("Medium Priority");
        MyThread low = new MyThread("Low Priority");

        high.setPriority(Thread.MAX_PRIORITY);      // 10
        medium.setPriority(Thread.NORM_PRIORITY);   // 5
        low.setPriority(Thread.MIN_PRIORITY);       // 1

        high.start();
        medium.start();
        low.start();
    }
}
```

OUTPUT:

```
Low Priority - Priority: 1 - Count: 1
Low Priority - Priority: 1 - Count: 2
Low Priority - Priority: 1 - Count: 3
Low Priority - Priority: 1 - Count: 4
High Priority - Priority: 10 - Count: 1
Low Priority - Priority: 1 - Count: 5
High Priority - Priority: 10 - Count: 2
High Priority - Priority: 10 - Count: 3
High Priority - Priority: 10 - Count: 4
High Priority - Priority: 10 - Count: 5
Medium Priority - Priority: 5 - Count: 1
Medium Priority - Priority: 5 - Count: 2
Medium Priority - Priority: 5 - Count: 3
Medium Priority - Priority: 5 - Count: 4
Medium Priority - Priority: 5 - Count: 5
```

3. Synchronized Method

```
class BankAccount {

    private int balance = 1000;

    synchronized void deposit(int amount) {

        balance = balance + amount;

        System.out.println(
            Thread.currentThread().getName()
            + " deposited Rs." + amount
            + " | Balance = Rs." + balance
        );
    }

    synchronized void withdraw(int amount) {

        if (balance >= amount) {

            balance = balance - amount;

            System.out.println(
                Thread.currentThread().getName()
                + " withdrew Rs." + amount
                + " | Balance = Rs." + balance
            );

        } else {

            System.out.println(
                Thread.currentThread().getName()
                + " - Insufficient Balance"
            );
        }
    }

    int getBalance() {
        return balance;
    }
}

class DepositThread extends Thread {

    BankAccount account;
```

```
    DepositThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.deposit(500);
    }
}

class WithdrawThread extends Thread {

    BankAccount account;

    WithdrawThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        account.withdraw(300);
    }
}

public class Main {

    public static void main(String[] args) throws Exception {

        BankAccount account = new BankAccount();

        Thread t1 = new DepositThread(account);
        Thread t2 = new WithdrawThread(account);
        Thread t3 = new DepositThread(account);
        Thread t4 = new WithdrawThread(account);

        t1.start();
        t2.start();
        t3.start();
        t4.start();

        t1.join();
        t2.join();
        t3.join();
        t4.join();

        System.out.println("\nFinal Balance = Rs." + account.getBalance());
    }
}
```

OUTPUT:

```
Thread-0 deposited Rs.500 | Balance = Rs.1500
Thread-1 withdrew Rs.300 | Balance = Rs.1200
Thread-3 withdrew Rs.300 | Balance = Rs.900
Thread-2 deposited Rs.500 | Balance = Rs.1400

Final Balance = Rs.1400
```

4. Synchronized Block

```
class TicketBooking {

    private int tickets = 5;

    void bookTicket(String name, int number) {

        synchronized (this) {

            if (tickets >= number) {

                System.out.println(
                    name + " is booking " + number + " ticket(s)."
                );

                tickets = tickets - number;

                System.out.println(
                    name + " booking successful."
                );

                System.out.println(
                    "Remaining Tickets = " + tickets
                );

            } else {

                System.out.println(
                    name + " - Not enough tickets available."
                );
            }
        }
    }
}

class Customer extends Thread {
```

```
    TicketBooking booking;
    String customerName;
    int tickets;

    Customer(TicketBooking booking, String customerName, int tickets) {
        this.booking = booking;
        this.customerName = customerName;
        this.tickets = tickets;
    }

    public void run() {
        booking.bookTicket(customerName, tickets);
    }
}

public class Main {

    public static void main(String[] args) throws Exception {

        TicketBooking booking = new TicketBooking();

        Customer c1 = new Customer(booking, "Rahul", 2);
        Customer c2 = new Customer(booking, "Arjun", 1);
        Customer c3 = new Customer(booking, "Prudhvi", 2);
        Customer c4 = new Customer(booking, "Kumar", 1);

        c1.start();
        c2.start();
        c3.start();
        c4.start();

        c1.join();
        c2.join();
        c3.join();
        c4.join();

        System.out.println("\nAll booking operations completed.");
    }
}
```

OUTPUT:

```
Rahul is booking 2 ticket(s).
Rahul booking successful.
Remaining Tickets = 3
Arjun is booking 1 ticket(s).
Arjun booking successful.
```

```
Remaining Tickets = 2
Prudhvi is booking 2 ticket(s).
Prudhvi booking successful.
Remaining Tickets = 0
Kumar - Not enough tickets available.

All booking operations completed.
```

5. Static Synchronization

```
class ExamCounter {

    static int counter = 0;

    static synchronized void updateCounter(String name) {

        counter++;

        System.out.println(
            name + " updated counter to " + counter
        );

        try {
            Thread.sleep(200);
        } catch (InterruptedException e) {
            System.out.println("Thread Interrupted.");
        }
    }
}

class Student extends Thread {

    String name;

    Student(String name) {
        this.name = name;
    }

    public void run() {
        ExamCounter.updateCounter(name);
    }
}

public class Main {

    public static void main(String[] args) throws Exception {
```

```
        Student s1 = new Student("Student 1");
        Student s2 = new Student("Student 2");
        Student s3 = new Student("Student 3");
        Student s4 = new Student("Student 4");
        Student s5 = new Student("Student 5");

        s1.start();
        s2.start();
        s3.start();
        s4.start();
        s5.start();

        s1.join();
        s2.join();
        s3.join();
        s4.join();
        s5.join();

        System.out.println("\nFinal Counter = " + ExamCounter.counter);
    }
}
```

OUTPUT:

```
Student 1 updated counter to 1
Student 2 updated counter to 2
Student 3 updated counter to 3
Student 4 updated counter to 4
Student 5 updated counter to 5

Final Counter = 5
```

6. Inter-Thread Communication

```
import java.util.Scanner;

class Customer {

    int AccountNo;
    String AccName;
    int Balance;

    Customer(int AccountNo, String AccName, int Balance) {
        this.AccountNo = AccountNo;
        this.AccName = AccName;
        this.Balance = Balance;
    }
```

```
synchronized void withdraw(int amount) {

    System.out.println("\nWithdraw requested: Rs." + amount);

    while (Balance < amount) {

        System.out.println(
            "Insufficient Balance. Current Balance = Rs."
            + Balance
        );

        System.out.println("Withdraw is waiting for deposit...");

        try {
            wait();
        } catch (InterruptedException e) {
            System.out.println("Withdraw Interrupted.");
            return;
        }
    }

    Balance = Balance - amount;

    System.out.println(
        "Withdrawal Successful: Rs." + amount
    );

    System.out.println(
        "Remaining Balance: Rs." + Balance
    );
}

synchronized void deposit(int amount) {

    Balance = Balance + amount;

    System.out.println(
        "\nDeposit Successful: Rs." + amount
    );

    System.out.println(
        "Current Balance: Rs." + Balance
    );

    notify();
```

```
    }
}

public class Main {

    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        Customer account =
            new Customer(1001, "Prudhvi", 1000);

        System.out.println("Account Number: " + account.AccountNo);
        System.out.println("Account Name: " + account.AccName);
        System.out.println("Initial Balance: Rs." + account.Balance);

        System.out.print("\nEnter Amount to Withdraw: ");
        int withdrawAmount = sc.nextInt();

        Thread withdrawThread = new Thread(() -> {
            account.withdraw(withdrawAmount);
        });

        withdrawThread.start();

        Thread.sleep(1000);

        System.out.print("Enter Amount to Deposit: ");
        int depositAmount = sc.nextInt();

        Thread depositThread = new Thread(() -> {
            account.deposit(depositAmount);
        });

        depositThread.start();

        withdrawThread.join();
        depositThread.join();

        System.out.println(
            "\nFinal Balance = Rs." + account.Balance
        );

        sc.close();
    }
}
```

OUTPUT:


Account Number: 1001
Account Name: Prudhvi
Initial Balance: Rs.1000
Enter Amount to Withdraw: